Damn Vulnerable Bank Penetration Testing Report

Prepared by: Charmaine Mangorima
Date: *December 11,2025*
Target Application: Damn Vulnerable Bank – API Layer
Test Environment: Local Vulnerable Bank Deployment
Testing Type: API Security Assessment (OWASP API Top 10 2023 and beyond)

1. Executive Summary

This report documents a penetration test conducted on the **Damn Vulnerable Bank** API with the objective of exploring **OWASP API Top 10 (2023) categories API4–API10**, including **CORS misconfiguration** and **path traversal** vulnerabilities. The testing environment intentionally contains weaknesses meant for training, but the exploited vulnerabilities reflect risks commonly found in real-world banking and fintech APIs.

Multiple high-severity issues were identified across the API surface, including:

- **API4 – Unrestricted Resource Consumption**
  Weak rate limiting and unbounded processes (example password reset attempts).

- **API5 – Broken Function-Level Authorization**
  Admin-only functions such as creating new admins and approving loans exposed to normal users when they manipulate tokens.

- **API6 – Unrestricted Access to Sensitive Business Flows**
  Abusable flows such as password reset, virtual card freeze/unfreeze, loan request
- **API7 – Server-Side Request Forgery (SSRF)**
  The URL-based image upload endpoint allows requests to internal services (metadata, internal admin routes).

- **API8 – Security Misconfiguration**
  Unsafe CORS, verbose debug messages.

- **API10 – Unsafe Consumption of APIs**
  External APIs (AI endpoint) processes unvalidated input leading to data leakage and prompt manipulation.

Additionally:

- **Path Traversal** in the file upload endpoint permits directory traversal through filename manipulation.

All these demonstrate severe systemic insecurity in sensitive banking operations.

2. Scope

In-Scope

- All API endpoints defined in openapi.json
  (example: */transactions/{account_number}*, */upload_profile_picture_url*, */admin/**, */api/virtual-cards/**)

- Features relevant to OWASP **API4–API10**

- File upload and download paths

- CORS behaviour

- JSON-based and query-parameter inputs

- Authorization/Authentication flows (only as they relate to API4–API10)

Out-of-Scope

- Frontend UI

- Internal source code analysis

3. Methodology

Steps

1. **Environment Setup:** Lab setup using python to start a local mock server in Cursor IDE.
2. **Spec Review:** OpenAPI/Swagger specification inspection to map endpoints.
3. **Endpoint Testing:** Targeted testing of endpoints for OWASP API4-API10,CORS and Path Traversal Vulnerabilities.

4. **Proof-of-Concept:** Tested vulnerabilities using test accounts
5. **Documentation:** Recorded findings and screenshots for each vulnerability.

Tools Used

- **Cursor IDE:** start the local mock server
- **Swagger Editor:** To get an overview of the endpoints
- **Burp Suite** – intercept & modify requests
- **Postman** – manual API requests
- **Browser DevTools** – confirm token flows
- **JWT.io** – decode tokens

4. Key Findings

Finding 1: Unrestricted Resource Consumption

OWASP Mapping: API4:2023

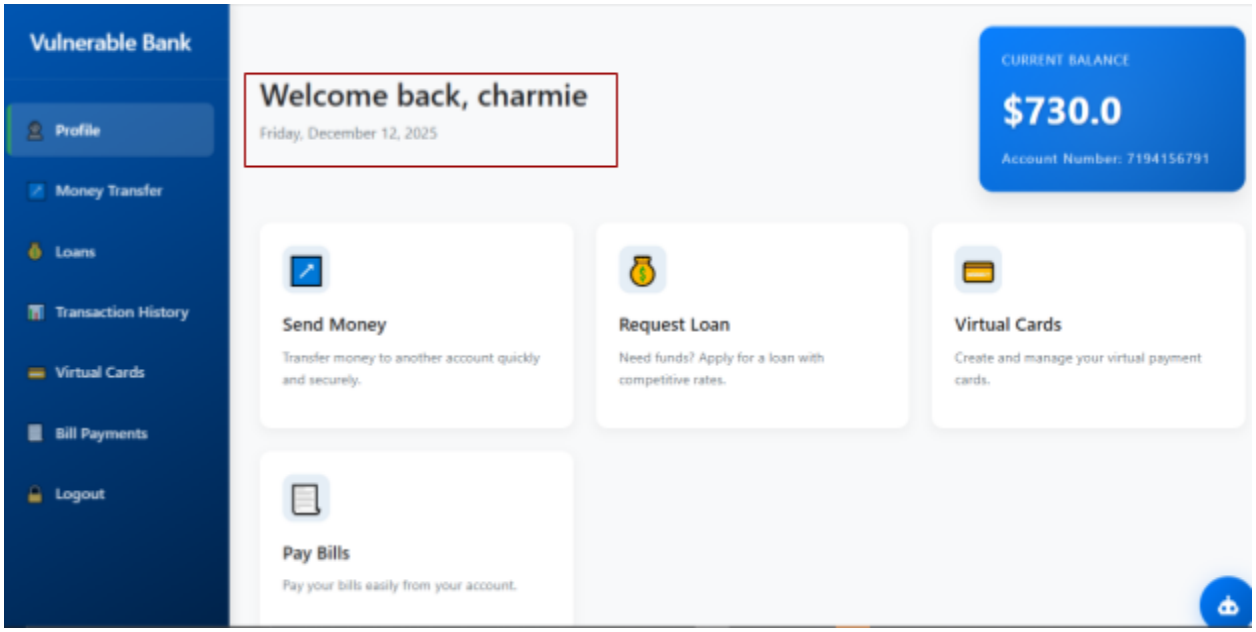
Description

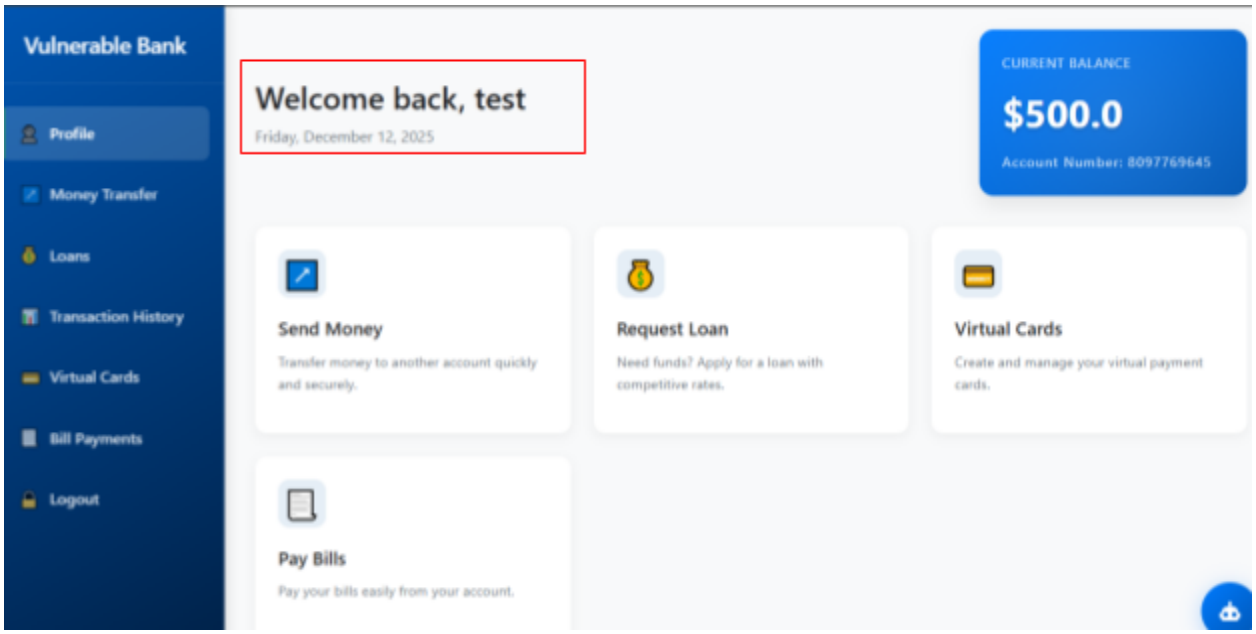
1. Several endpoints lack rate limiting, especially password reset flows:
   - ***/api/v1/forgot-password***
   - ***/api/v1/reset-password***
   - ***/upload_profile_picture***

Attackers can continuously request password resets or brute-force short PINs (3–4 digits), causing resource exhaustion or account takeover attempts.

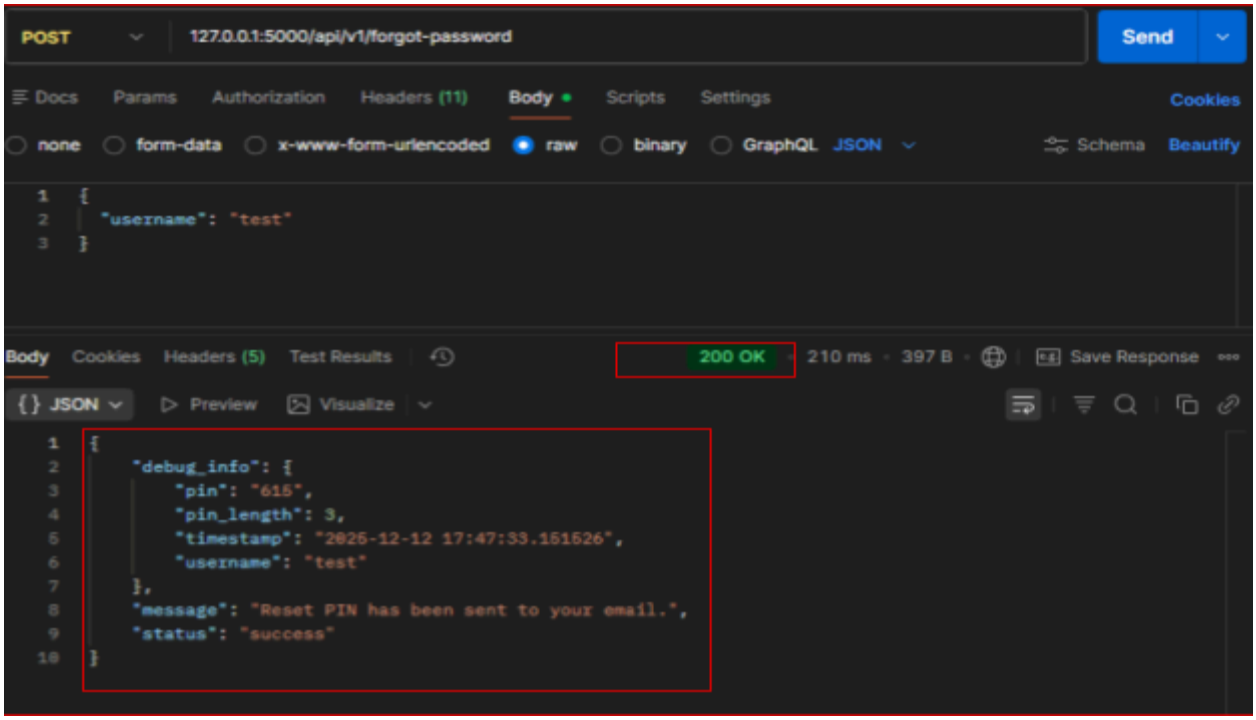
Evidence

I have test users registered from Week2 Pentesting.

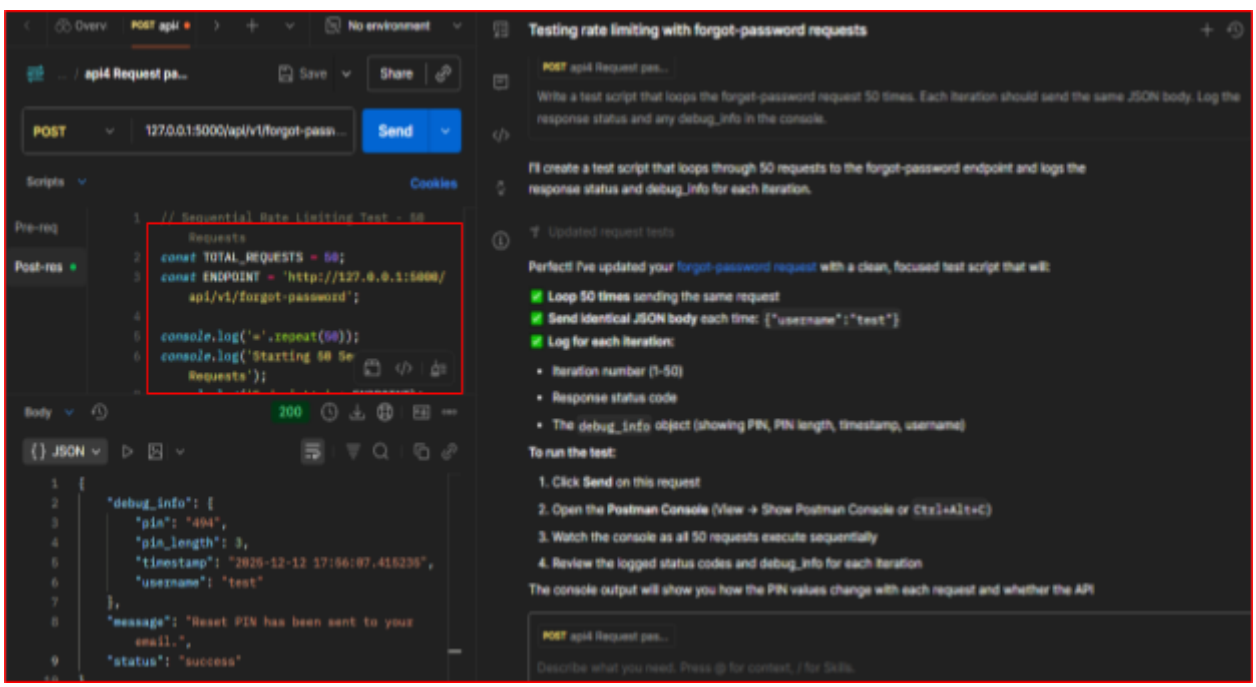




First I sent a single request to the ***/api/v1/forgot-password*** endpoint and I saw the structure of the response.



After this I asked the AI agent in Postman to create a script that sends 50 consecutive requests to this endpoint and the script was successfully created.



I sent the request again and all 50 requests got responses within a second.

```
› POST http://127.0.0.1:5000/api/v1/forgot-password                                  200  45 ms
"Iteration 47 | Status: 200"
"  debug_info:"  "{"pin":"128","pin_length":3,"timestamp":"2025-12-12 17:56:09.431238","username":"test"}"
› POST http://127.0.0.1:5000/api/v1/forgot-password                                  200  157 ms
"Iteration 48 | Status: 200"
"  debug_info:"  "{"pin":"485","pin_length":3,"timestamp":"2025-12-12 17:56:09.588238","username":"test"}"
› POST http://127.0.0.1:5000/api/v1/forgot-password                                  200  19 ms
"Iteration 49 | Status: 200"
"  debug_info:"  "{"pin":"613","pin_length":3,"timestamp":"2025-12-12 17:56:09.608239","username":"test"}"
› POST http://127.0.0.1:5000/api/v1/forgot-password                                  200  13 ms
"Iteration 50 | Status: 200"
"  debug_info:"  "{"pin":"356","pin_length":3,"timestamp":"2025-12-12 17:56:09.675238","username":"test"}"
"================================================"
"Completed all 50 requests"
"================================================"
```

This proves the endpoint lacks:

- rate limiting
- brute-force protection

Risk

- PIN brute-forcing becomes easy
- Attackers can trigger mass reset requests
- Denial of service on the reset endpoint

Mitigation

- Apply rate limiting (example 3 requests/min per IP)
- Use long, cryptographically secure reset tokens
- Remove debug_info from responses

2. The /upload_profile_picture end point is vulnerable to Unrestricted Resource Consumption because there is no file size limit for the image being uploaded. This leads to resource exhaustion and probably system failure when exploited by attackers.

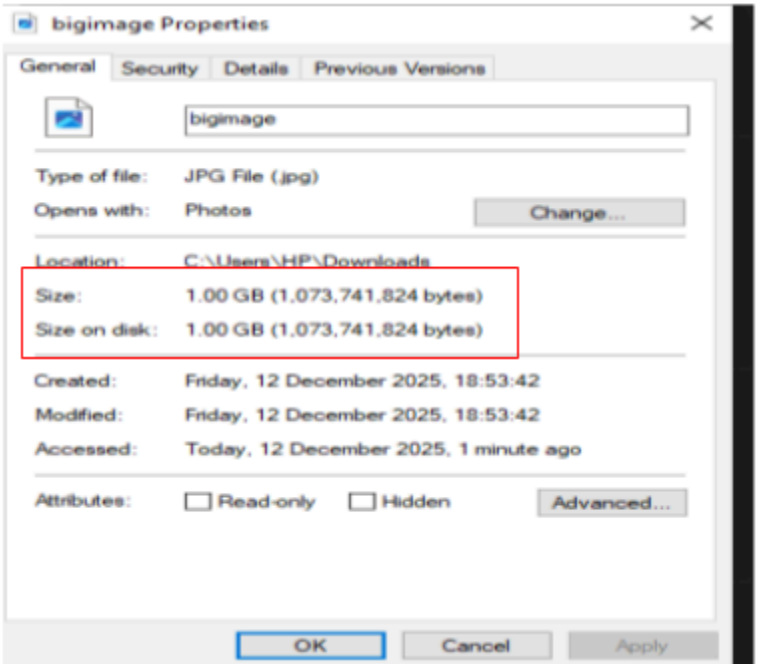
Evidence

I created a large image file that is 1gb in size

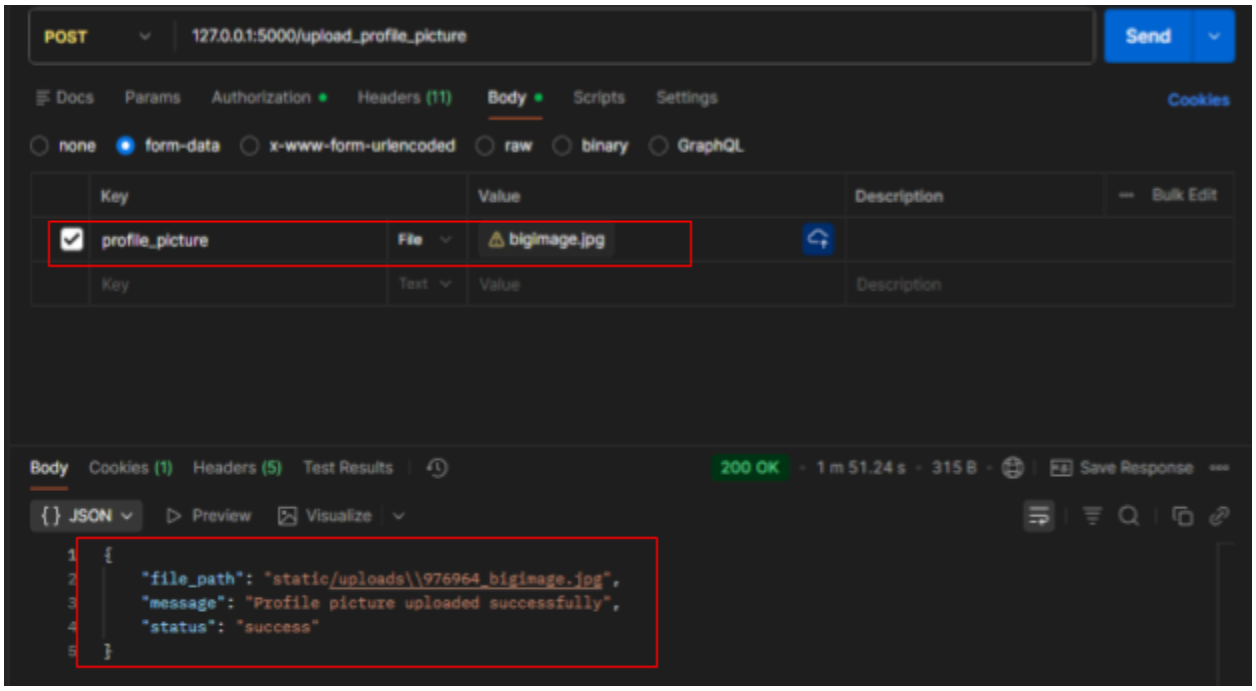
```
C:\Users\HP\Downloads>fsutil file createnew bigimage.jpg 500000000
File C:\Users\HP\Downloads\bigimage.jpg is created

C:\Users\HP\Downloads>fsutil file createnew bigimage.jpg 1073741824
File C:\Users\HP\Downloads\bigimage.jpg is created

C:\Users\HP\Downloads>_
```



I sent a request to change “test” user’s profile picture and i used “bigimage”. The profile picture was uploaded successfully.



Risks:

1. High server CPU usage
2. High Memory (RAM) usage
3. Disk exhaustion
4. Denial of Service (DoS)

Mitigation

- Use file size limit for file upload endpoints
- Apply rate limiting

Finding 2: Broken Function-Level Authorization

OWASP Mapping: API5:2023

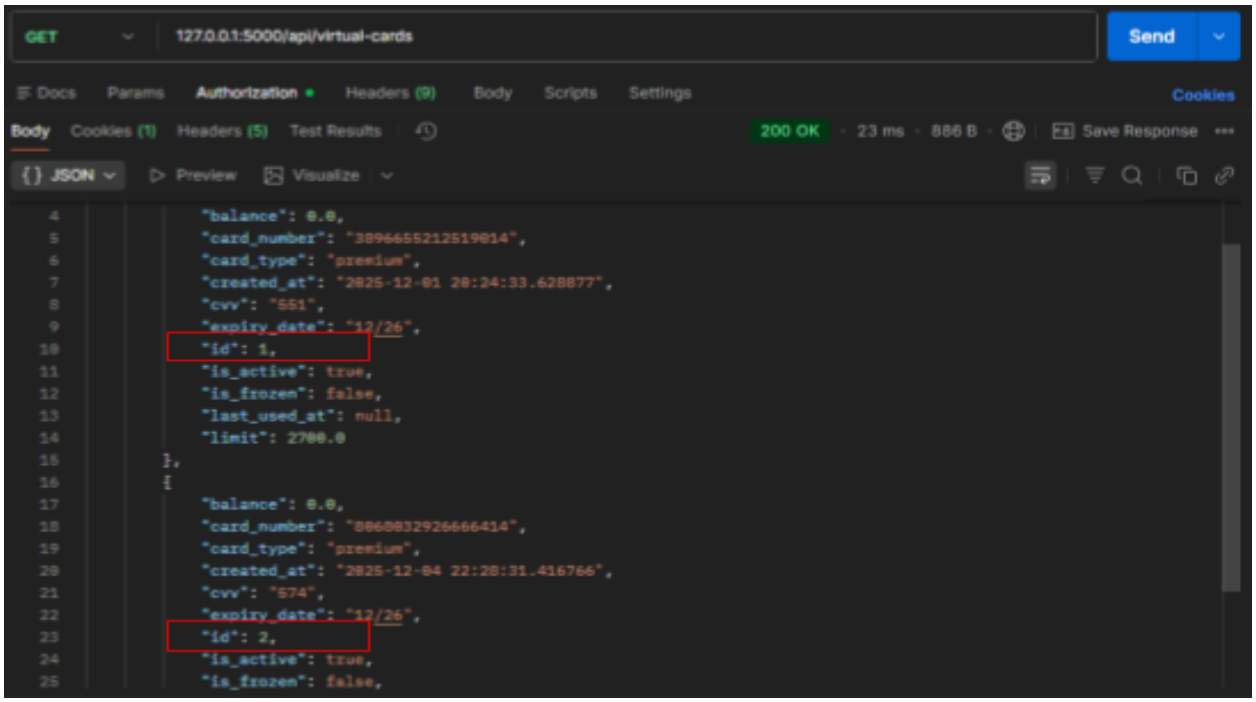
BFLA is when an API does not check authorization when a user has access to another user's data and they are able to modify or delete it including requests that require admin priviledges.

Evidence

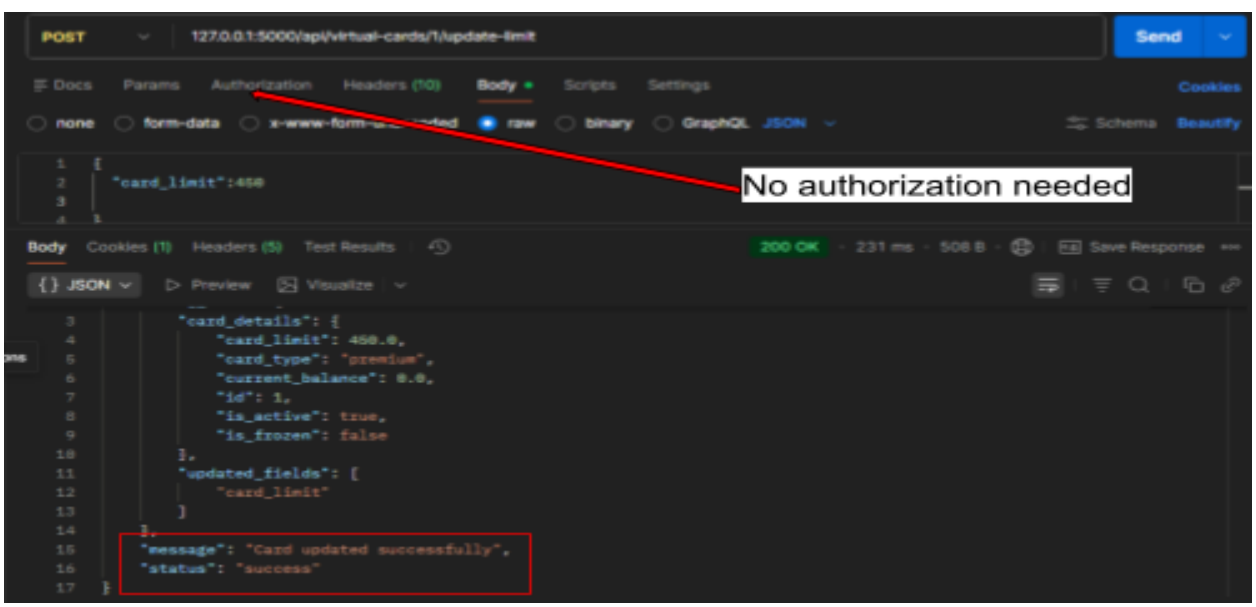
1. The ***/api/virtual-cards/{card_id}/update-limit*** lets any user update another user's card limit as long as they know the card id and that is BFLA.

I was not logged in(no token provided) but I updated "charmie's" virtual card limit(card_id=1).

Charmie has 2 virtual cards:



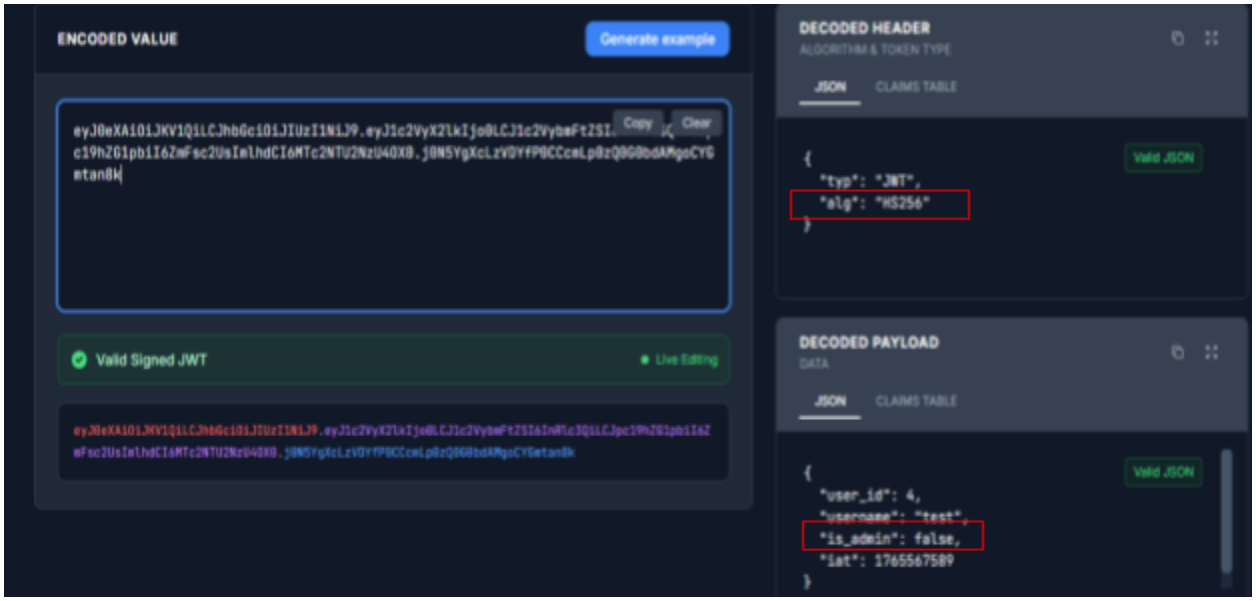
I updated the card limit and I was not even logged in



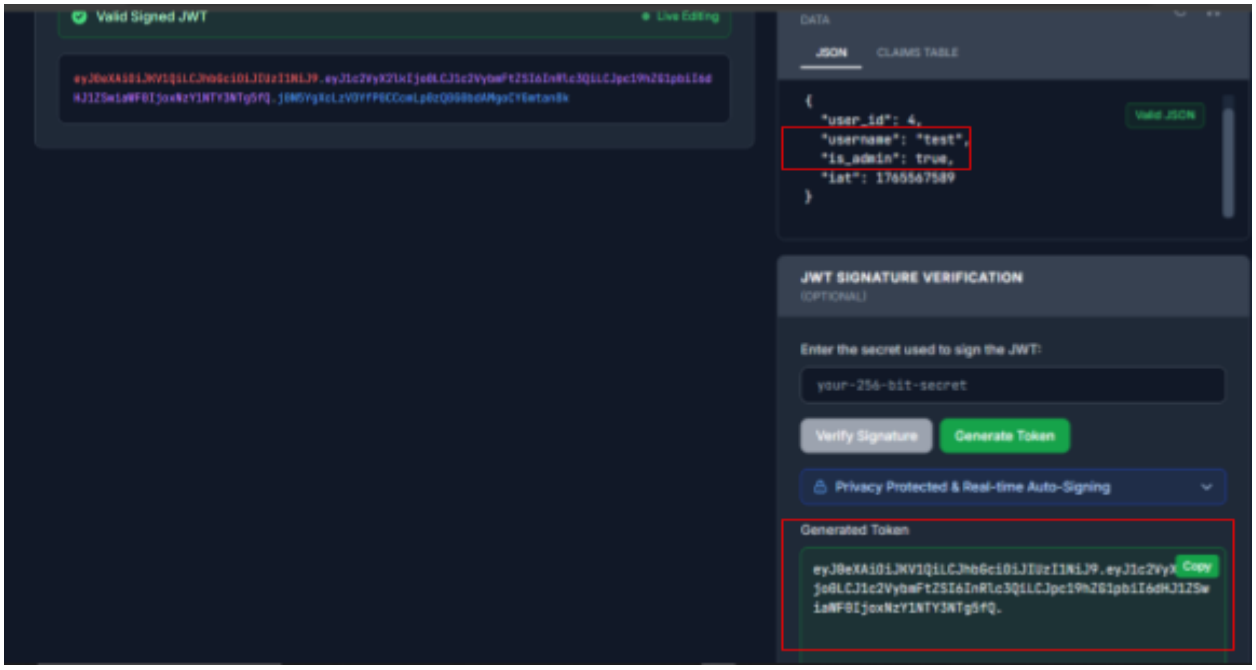
2. The next POC involves privilege escalation using the tokens provided by the API which are vulnerable. Using xjwt.io, I inspected and tempered around with “test’s” token.

OWASP Mapping : API2 Broken Authentication and API5 BFLA

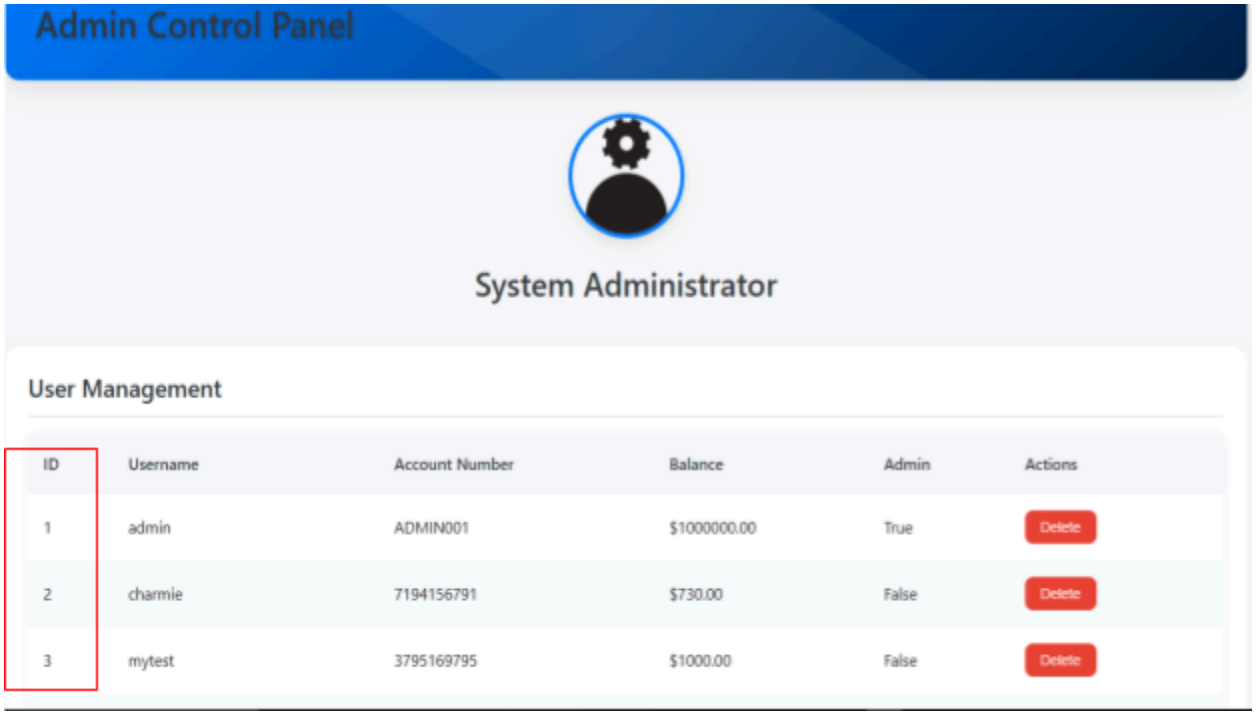
Using xjwt.io, I inspected the token and it is missing exp claim, it is signed using a weak algorithm and it has the “is_admin” claim hard-coded.



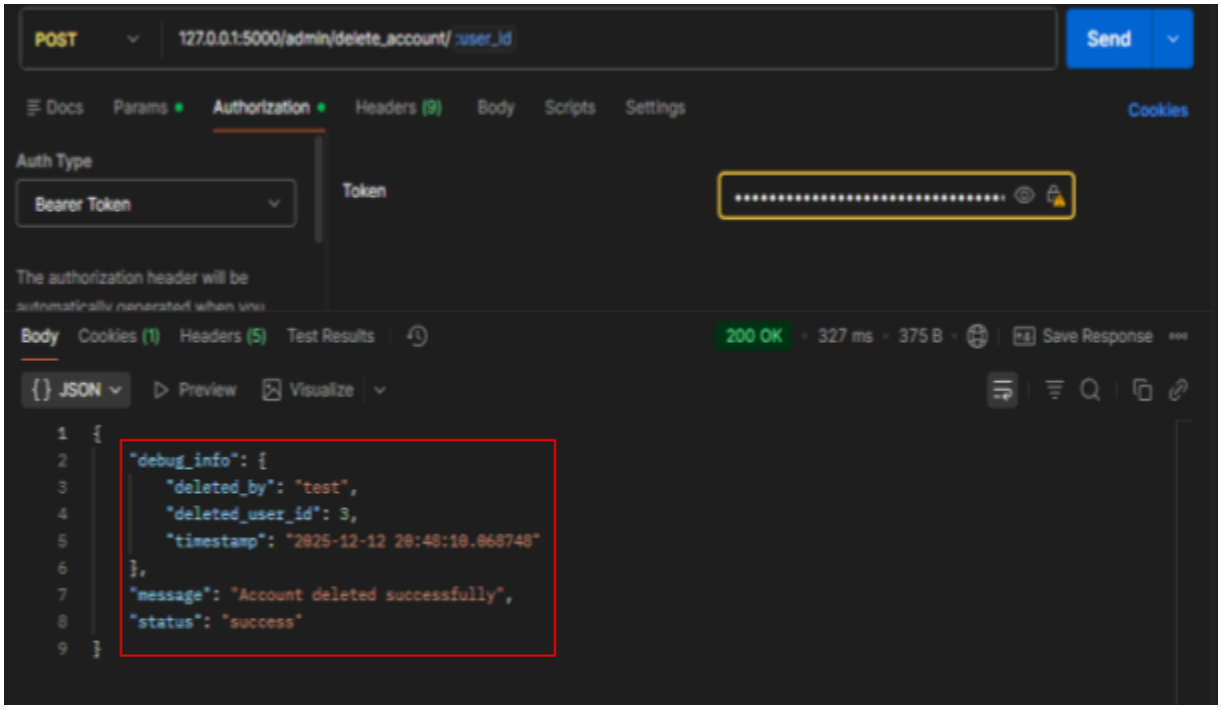
I modified the admin claim to true(making “test” an admin and got a new token.



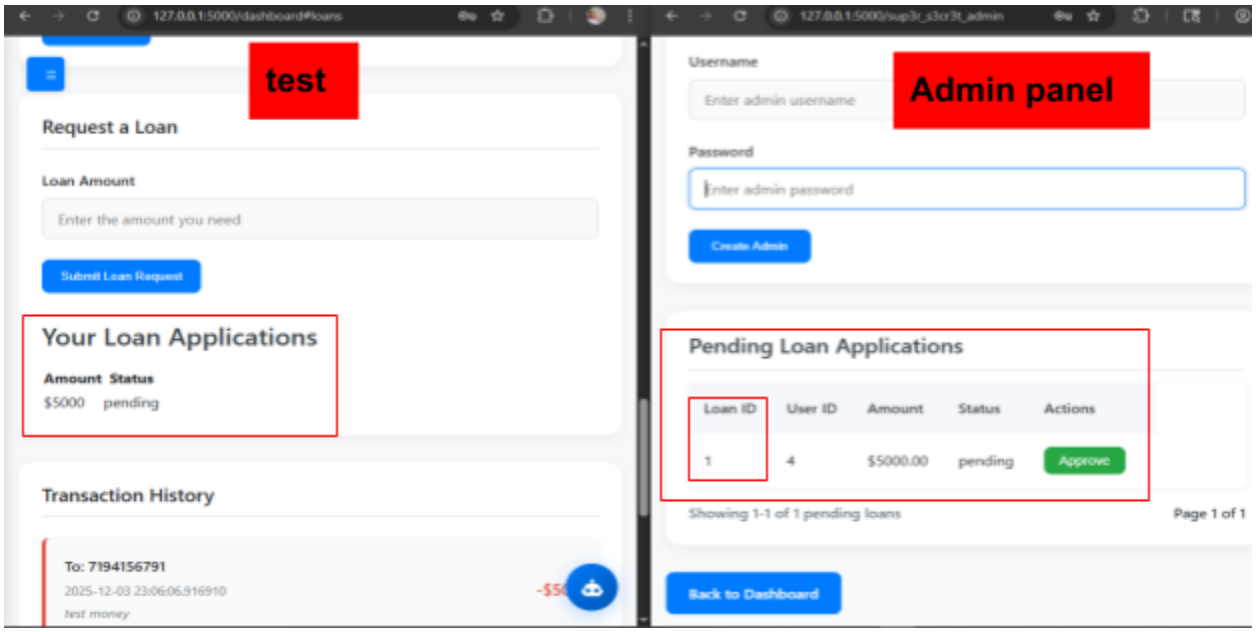
Using this new token I was able to access the ***/sup3r_s3cr3t_admin*** endpoint and took note of user ids.



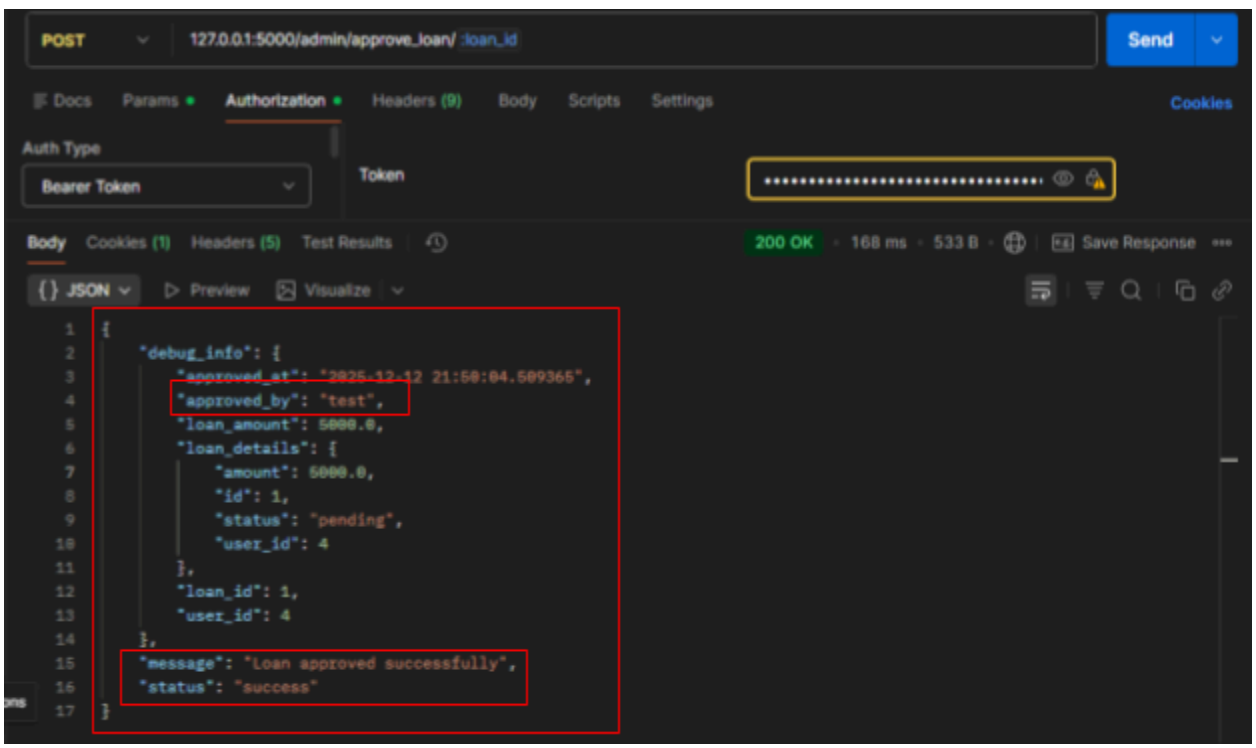
a. I then went ahead and accessed the ***/admin/delete_account/{user_id}*** endpoint and deleted a user(requires admin privileges).



b. I went into the normal “test” user account and requested a loan.



Using the forged JWT for “test” (admin escalated), test approved their own loan using the ***/admin/approve_loan/:loan_id*** endpoint.



Risk

- Unauthorized users can create admins
- Unauthorized users can modify other people's card limits or even freeze cards.
- Delete accounts
- Approve loans

  This represents total privilege escalation.

Mitigation

- Implement strict Role Based Access Control (RBAC)
- Implement authorization checks for all sensitive endpoints
- Validate role in server before any admin logic
- Maintain audit logs for all admin actions
- Use strong signing algorithms for JWTs and practice secure token creation

Finding 3: Unrestricted Access to Sensitive Business Flows

OWASP Mapping: API6:2023

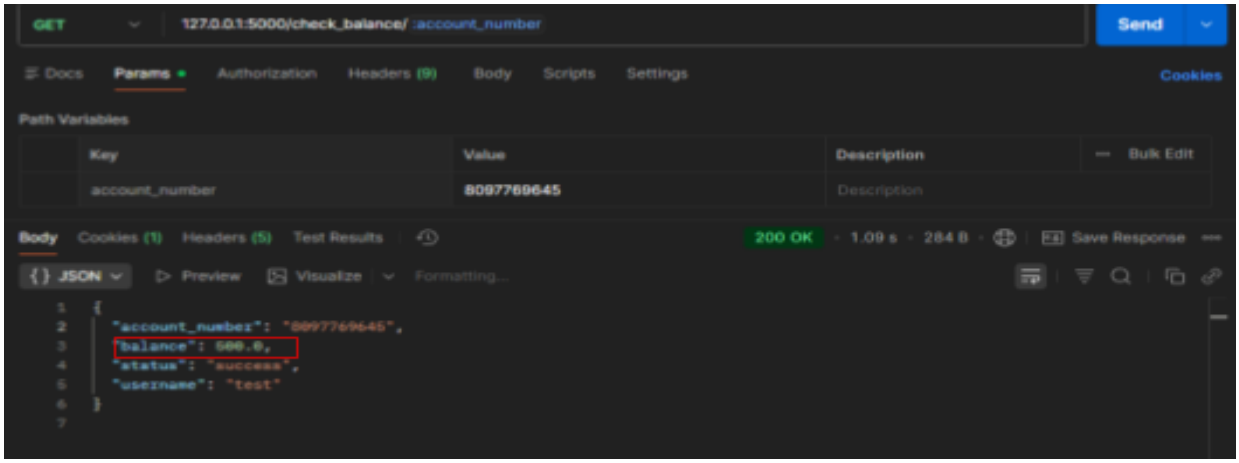
Unrestricted Access to Sensitive Business Flows is a critical vulnerability that enables attackers to exploit APIs by gaining excessive access to sensitive business processes. This vulnerability occurs when API endpoints expose critical flows without appropriate access restrictions, leading to potential harm to the business.

Attackers leverage their understanding of the target business model and automate access to sensitive flows, causing various forms of harm. Lack of a clear view of the API and inadequate protection mechanisms contribute to the exploitation of this vulnerability.

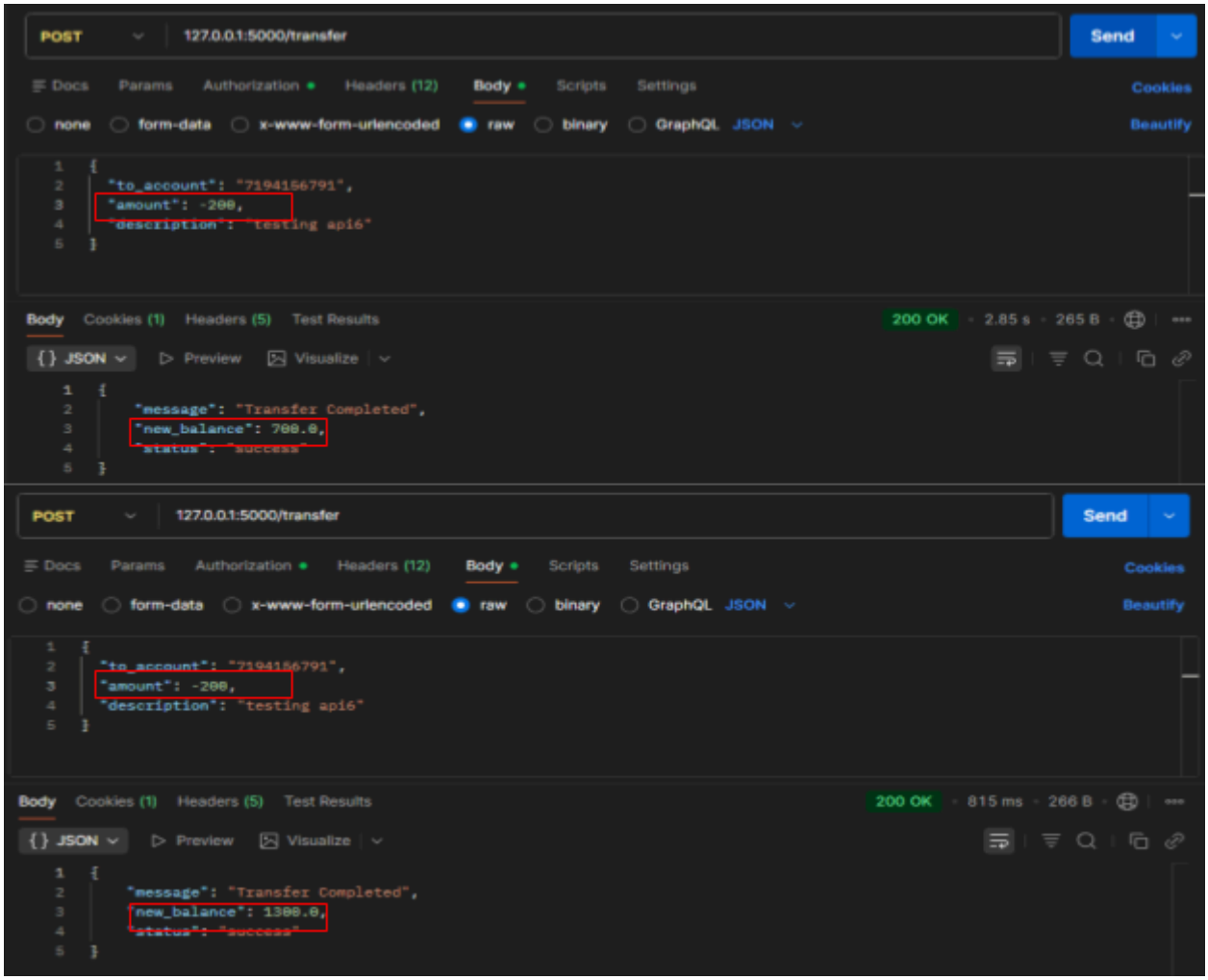
Evidence

1. The ***/transfer*** endpoint allows a user to transfer a negative amount and it results in debiting the user's account therefore increasing balance, as long as the positive value amount being transfered is less than the balance.

“test” has $500 in the account



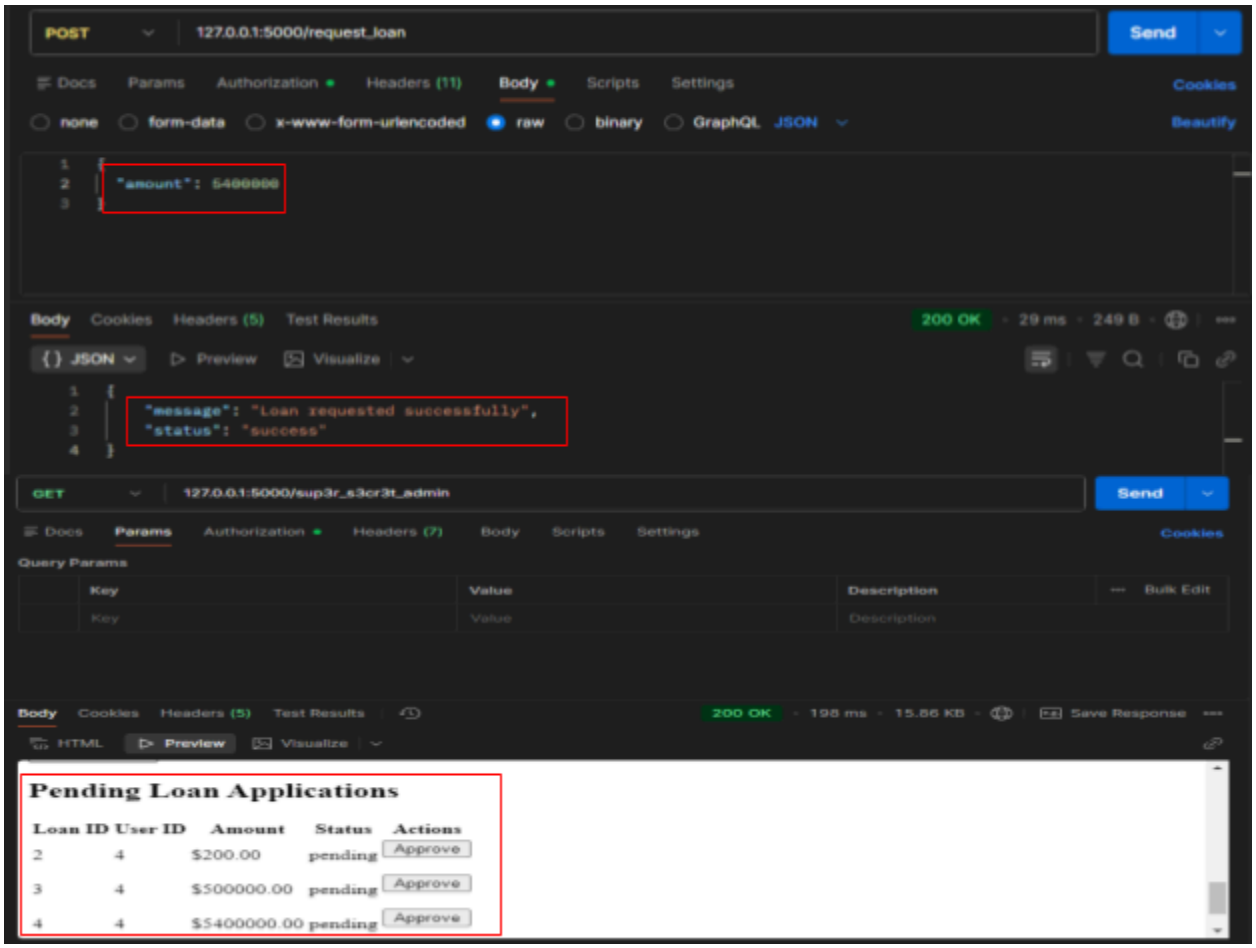
I transferred $-200 every time and the balance kept increasing



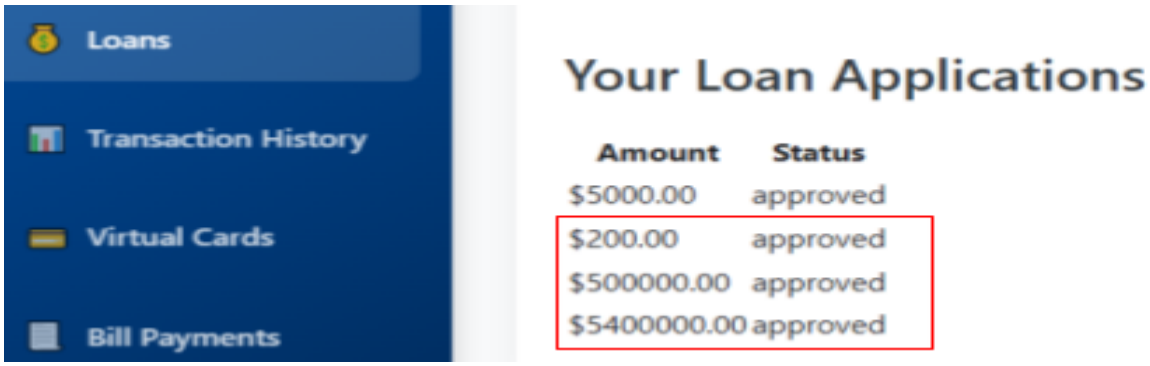
This shows how an attacker can abuse the normal business logic of transferring money because there is lack of validation on the endpoint

2. The ***/request_loan*** endpoint allows a user to request loans as many times as possible without checking if the user already has other loan requests.

I requested for 3 loans consecutively using “test” account:



Using the forged admin token, “test” approved all the loans



This shows how attackers can abuse the normal business logic by requesting multiple loans.

Risk

- Transaction integrity compromised
- Opportunity for fraud

Mitigation

- Enforce transactional integrity (locking)
- Include contextual validation
- Validate and put limits for loan requests

Finding 4: Server-Side Request Forgery (SSRF)

OWASP Mapping: API7:2023

Endpoint */upload_profile_picture_url*

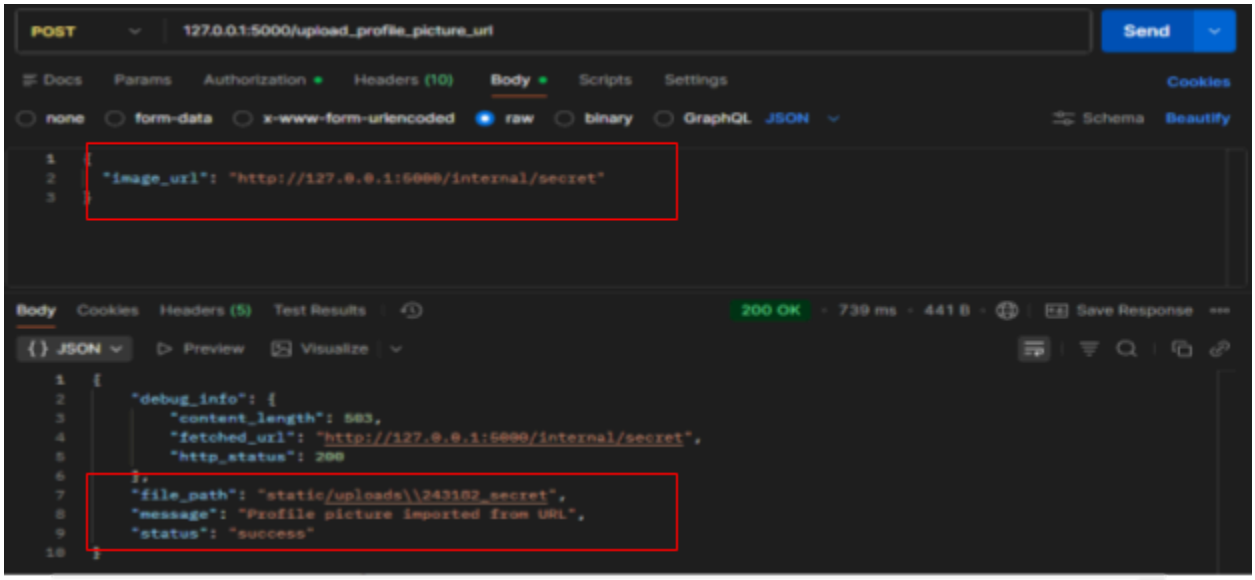
Server-Side Request Forgery (SSRF) flaws occur whenever an API is fetching a remote resource without validating the user-supplied URL. It allows an attacker to coerce the application to send a crafted request to an unexpected destination.

The server fetches any user-given URL without validation:

- Follows redirects
- Accepts internal/private IPs
- No SSL verification
- No allowlist
- No content-type checks

Evidence

I tricked the server into returning internal secrets by passing a url that is not of an image by a url to secrets and I successfully accessed the secret.



Risk

- Internal admin endpoints accessible
- Cloud metadata exposure
- Possible RCE if internal services are vulnerable

Mitigation

- Implement host/IP allowlist

- Block all RFC1918/private ranges
- Verify SSL certificates
- Reject non-image content types

Finding 5: Security Misconfiguration (CORS + Debug Info + Unsafe Uploads)

OWASP Mapping: API8:2023

Endpoints :

- */debug/users*
- */api/v{version}/forgot-password*
- */api/transactions?account_number=string*

Description

A security misconfiguration arises when essential security settings are either not implemented or implemented with errors. Security misconfiguration can take many forms, such as not changing default passwords, not patching systems with security updates or leaving unnecessary ports open or giving out sensitive information in errors. Hackers can exploit these misconfigurations to gain access to sensitive data or to take control of systems.

1. Debug fields such as debug_info exposed
2. File upload endpoint does not sanitize filenames or types
3. Usernames and passwords are stored as plain text, they must be encrypted.
4. The Bank API is publicly available
5. The passwords are not encrypted and they are weak in all versions and can be bruteforced easily
6. Detailed error messages

Evidence

1. Misconfigured CORS

CORS (Cross-Origin Resource Sharing) controls which websites are allowed to read API responses from a browser.

A CORS misconfiguration occurs when:

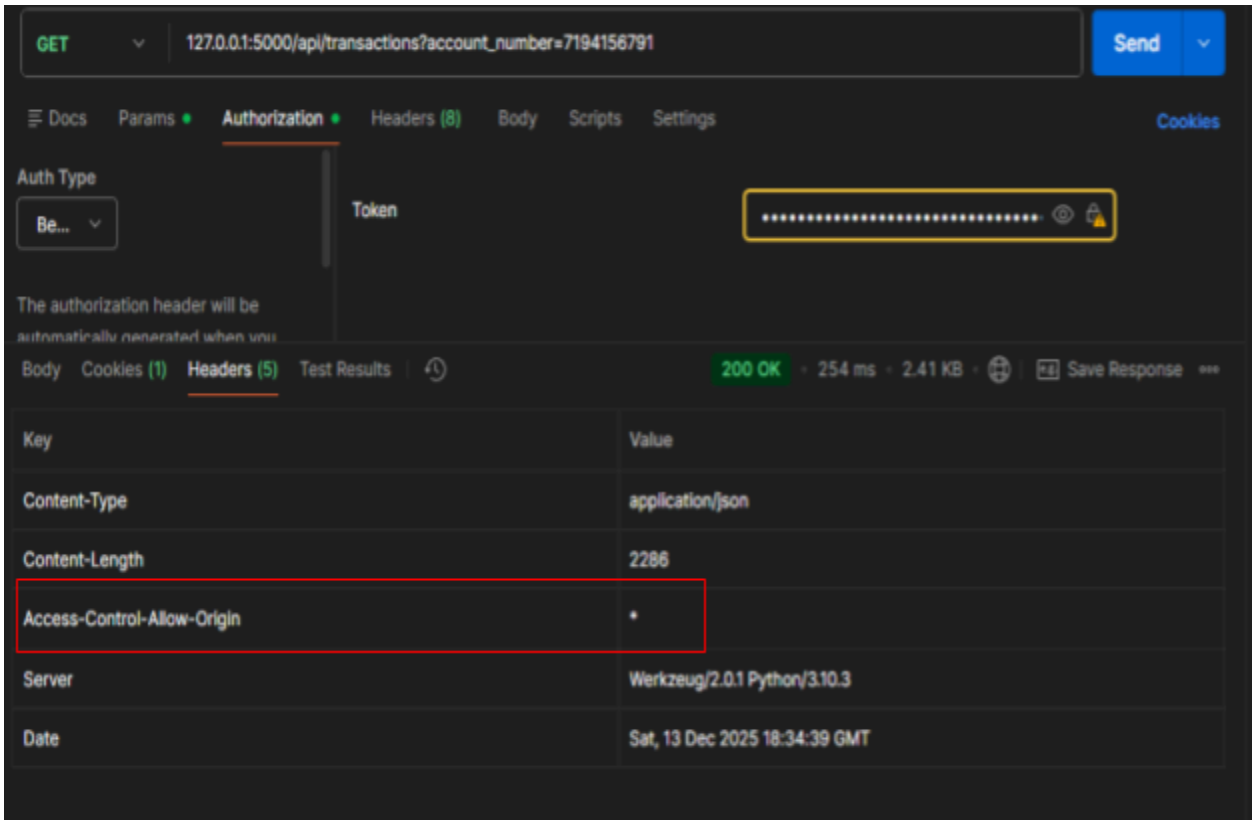
- The API allows any origin (*)
- The API reflects the Origin header
- Credentials are allowed with unsafe origins
- Sensitive endpoints respond to untrusted origins

The API implements an insecure CORS policy that allows responses to be shared with untrusted origins. By supplying a malicious Origin header, the API responds with permissive CORS headers, enabling any external website to read sensitive API responses.

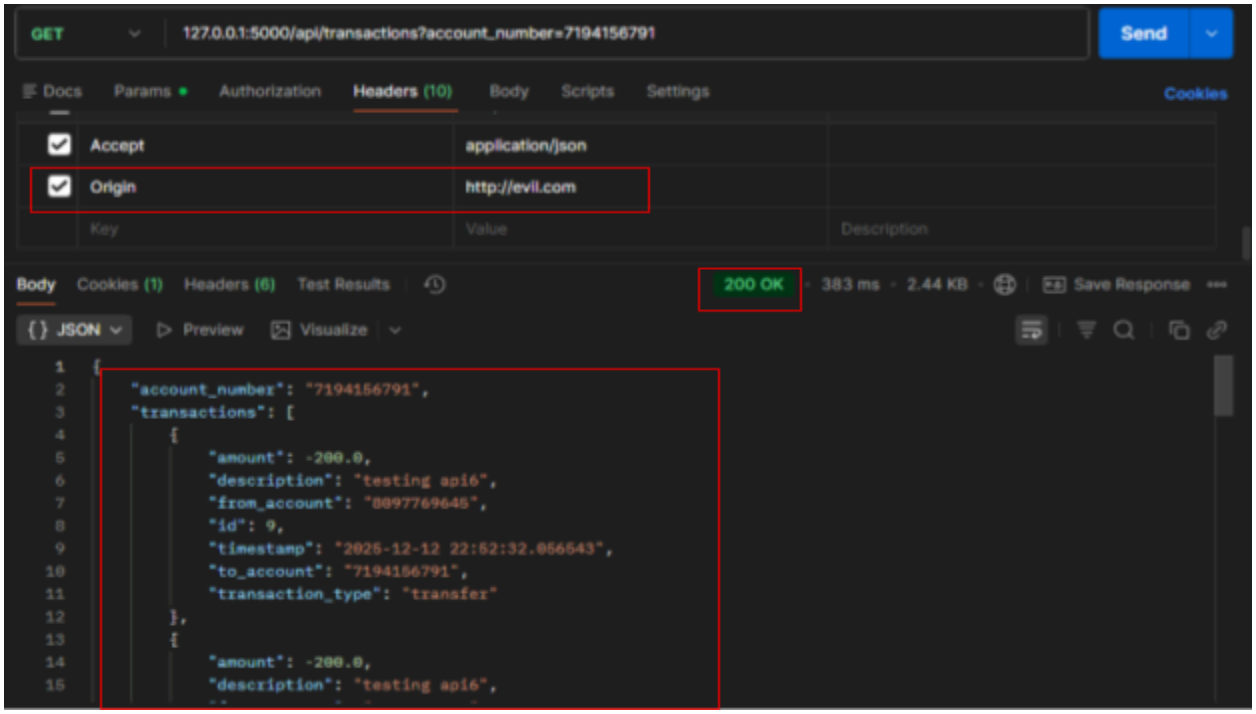
This misconfiguration violates the principle of least privilege and exposes authenticated API data to cross-origin attacks.

<u>POC</u>

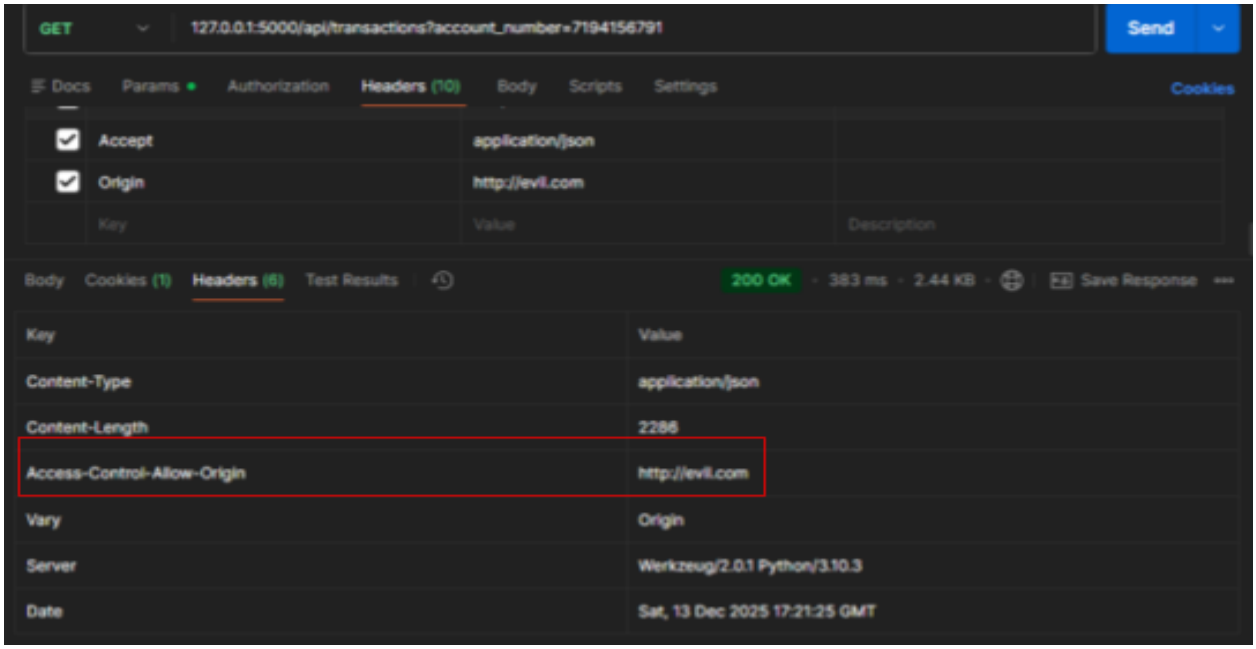
I checked the response headers on the ***/api/transactions?account_number=string*** endpoint and they are set to allow all origins.



I sent a GET request to the ***/api/transactions?account_number=string*** and added a malicious origin to the request header and the API returned the expected response.



The API responds with **Access-Control-Allow-Origin: http://evil.com**, indicating that an untrusted, attacker-controlled origin is explicitly permitted to read API responses. This misconfiguration allows malicious websites to access sensitive data via a victim's authenticated browser session.

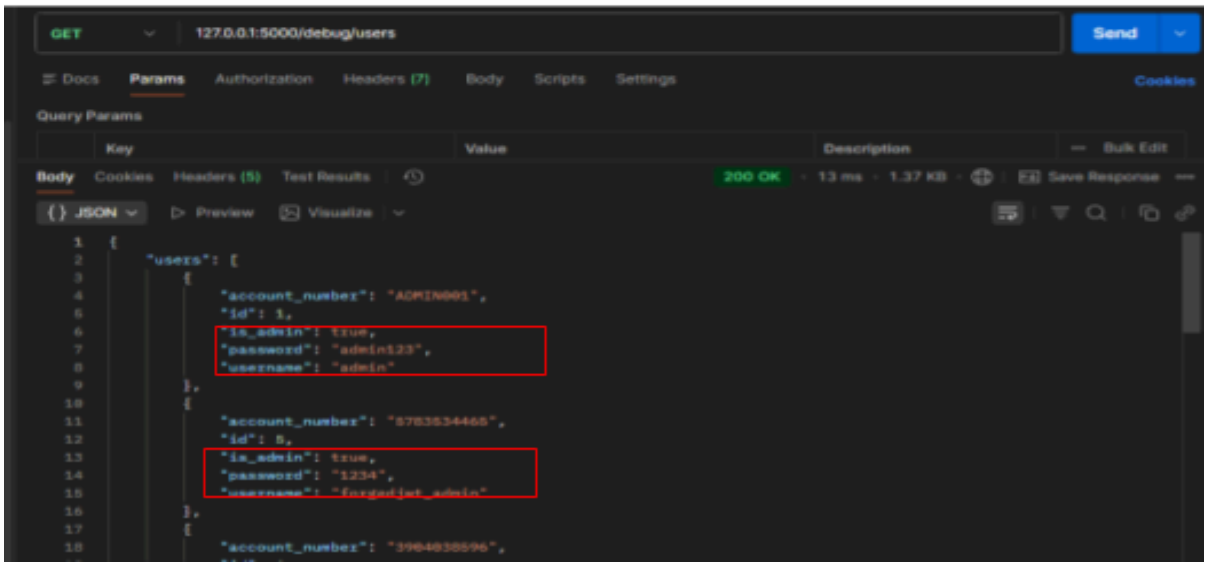


Mitigation

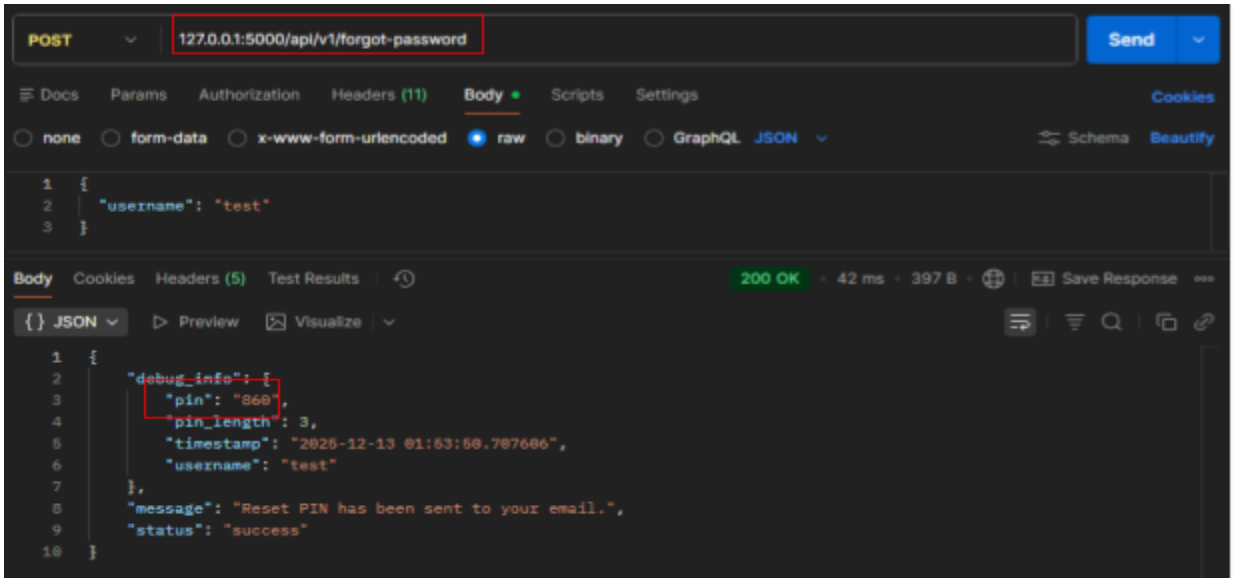
- Manage CORS centrally via an API gateway or reverse proxy.
- Regularly audit CORS configurations during deployment.
- Include automated security checks to detect permissive CORS headers.
- Treat CORS configuration as part of the application's security baseline.

2. While I was looking for security misconfigurations, I noticed that the developer left a debug in the app.py file that is not documented in the API. The ***/debug/users*** endpoint exposes all user information without even authenticating the user accessing the data. The information exposed includes usernames and passwords for all users registered on the app.

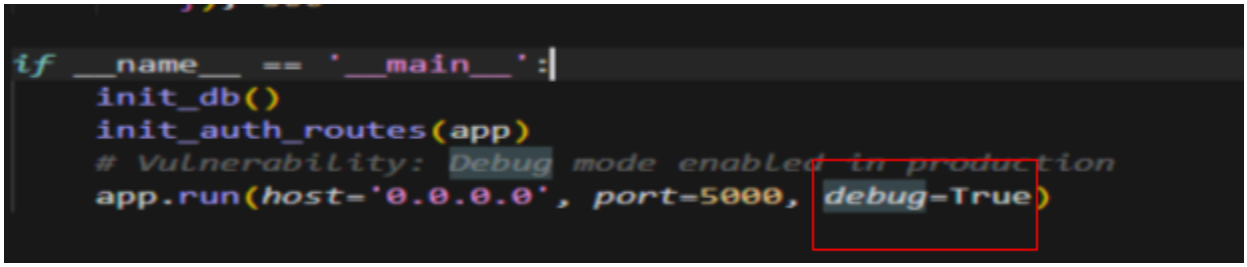
```
@app.route('/debug/users')
def debug_users():
    users = execute_query("SELECT id, username, password, account_number, is_admin FROM users")
    return jsonify({'users': [
        {
            'id': u[0],
            'username': u[1],
            'password': u[2],
            'account_number': u[3],
            'is_admin': u[4]
        } for u in users
    ]})
```



3. Weak pins that are exposed and not not encrypted can be bruteforced in the ***/api/v1/forgot-password*** endpoint:



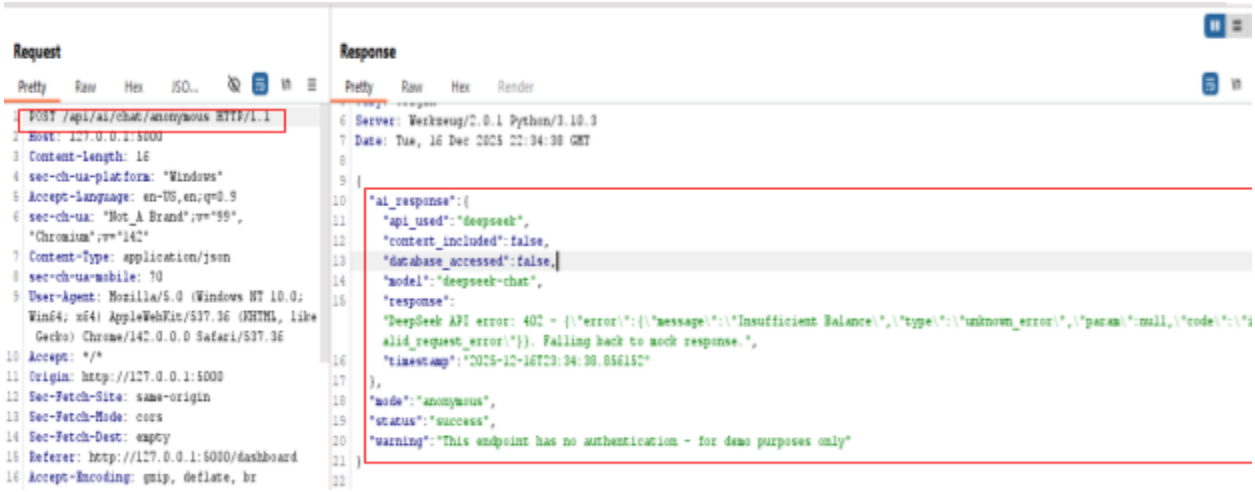
4. Debug mode is enabled in production



5. Detailed error messages

The api/ai/chat endpoints return detailed error messages:

- Internal third-party API errors are returned directly to the client
- Billing status of an external service is exposed
- Error handling is overly verbose (debug-style output)



Request

Pretty Raw Hex JSO...

```
1 POST /api/ai/chat/anonymous HTTP/1.1
2 Host: 127.0.0.1:5000
3 Content-Length: 16
4 sec-ch-ua-platform: "Windows"
5 Accept-Language: en-US,en;q=0.9
6 sec-ch-ua: "Not_A Brand";v="99",
  "Chromium";v="142"
7 Content-Type: application/json
8 sec-ch-ua-mobile: ?0
9 User-Agent: Mozilla/5.0 (Windows NT 10.0;
  Win64; x64) AppleWebKit/537.36 (KHTML, like
  Gecko) Chrome/142.0.0.0 Safari/537.36
10 Accept: */*
11 Origin: http://127.0.0.1:5000
12 Sec-Fetch-Site: same-origin
13 Sec-Fetch-Mode: cors
14 Sec-Fetch-Dest: empty
15 Referer: http://127.0.0.1:5000/dashboard
16 Accept-Encoding: gzip, deflate, br
```

Response

Pretty Raw Hex Render

```
6 Server: Werkzeug/2.0.1 Python/3.10.3
7 Date: Tue, 16 Dec 2025 22:34:38 GMT
8
9 {
10   "ai_response":{
11     "api_used":"deepseek",
12     "content_included":false,
13     "database_accessed":false,
14     "model":"deepseek-chat",
15     "response":
       "DeepSeek API error: 402 - {\"error\":{\"message\":\"Insufficient Balance\",\"type\":\"unknown_error\",\"param\":null,\"code\":\"i
       alid_request_error\"}}. Falling back to mock response.",
16     "timestamp":"2025-12-16T23:34:38.856152"
17   },
18   "mode":"anonymous",
19   "status":"success",
20   "warning":"This endpoint has no authentication - for demo purposes only"
21 }
22
```

Risk

- **Immediate Account Compromise**
  Attackers can log in directly using leaked credentials without any brute-force effort.

- **Privilege Escalation**
  If admin credentials are exposed, attackers gain full control over the system.

- **Credential Reuse Attacks**
  Users often reuse passwords across platforms, allowing attackers to compromise external services.

- **Data Protection and Compliance Violations**
  Exposing passwords violates security best practices and regulations such as GDPR and PCI-DSS (in a banking context)

- **Complete Trust Breakdown**
  Any attacker accessing this endpoint can impersonate users, manipulate transactions or extract confidential financial data.

Mitigation

- Disable or remove all debug endpoints from non-development environments

- Block access to /debug or similar routes at the routing level

- Error messages should return only necessary information and not overly detailed

- Encrypt pins and sensitive information and avoid exposing it in debug information

Finding 6: Improper API Inventory Management

OWASP Mapping: API9:2023 – Improper Inventory Management

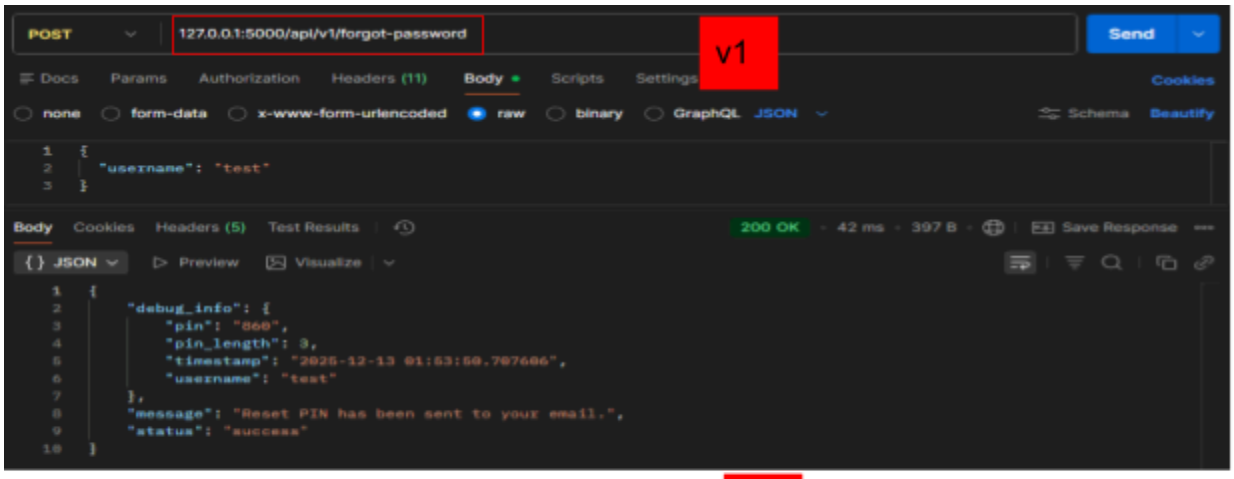
Description

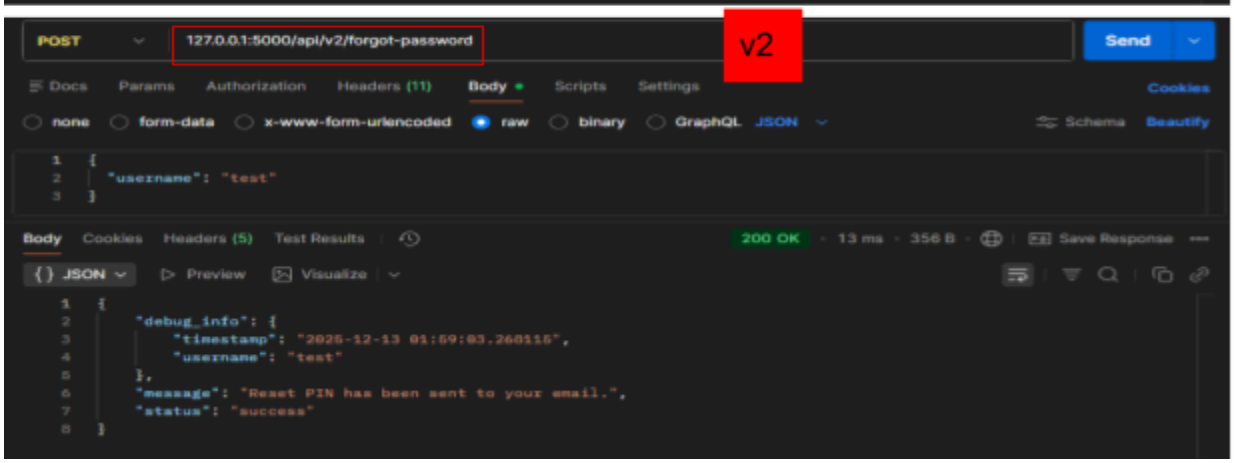
The application exposes multiple undocumented, debug and internal endpoints that are not intended for public use. These endpoints indicate poor API lifecycle and inventory management, allowing attackers to discover and interact with endpoints that should have been disabled, restricted or removed before production deployment.

Evidence

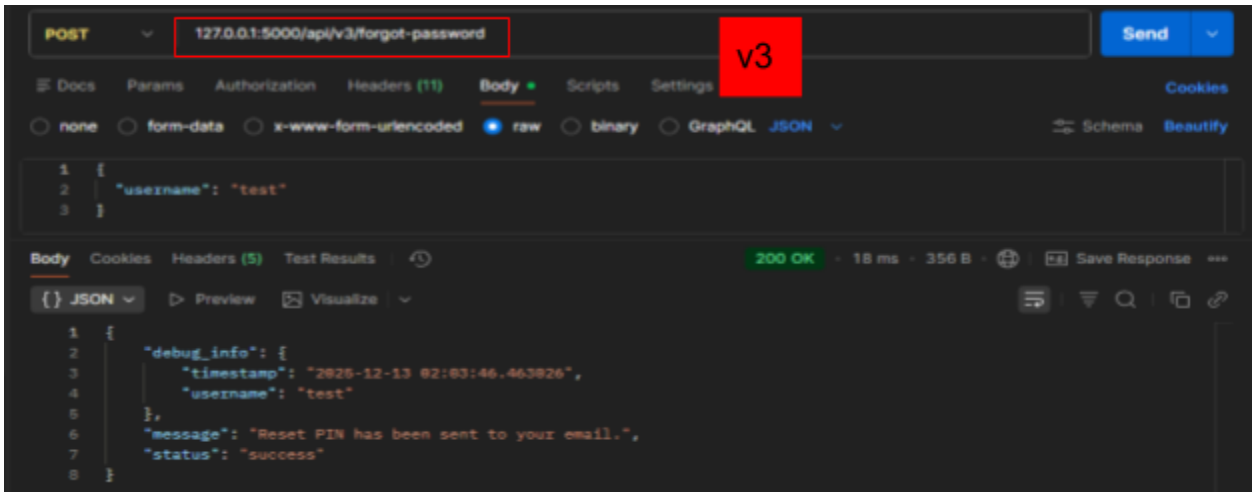
When I exploited the ***/api/v{version}/forgot-password (check finding 1),*** I used v1 which exposed debug information including the pin.

I used v2 and it worked





I also used v3 and it worked.



This leaves room for attackers to exploit vulnerabilities in the old versions(v1 and v2) that are still left in the public.

Risk

Leaving old API versions accessible introduces the following risks:

1. **Reintroduction of Fixed Vulnerabilities**
   Older versions may have vulnerabilities that are fixed in new version

2. **Security Control Bypass**
   Attackers may intentionally target older versions to bypass protections implemented in newer APIs.

3. **Expanded Attack Surface**
   Each active version increases the number of exploitable endpoints.

4. **Inconsistent Security Policies**
   Different versions may apply different authentication or authorization logic, leading to privilege escalation.

5. **Maintenance and Monitoring Gaps**
   Legacy APIs are often not monitored, logged or patched.

Mitigation

- Maintain a complete and up-to-date API inventory

- Implement formal API lifecycle management.
- Use an API gateway to enforce versioning and access control

Finding 7: Unsafe Consumption of APIs

OWASP Mapping: API10:2023 –Unsafe Consumption of APIs

Endpoints:

- /api/ai/chat
- /api/ai/chat/anonymous

Description

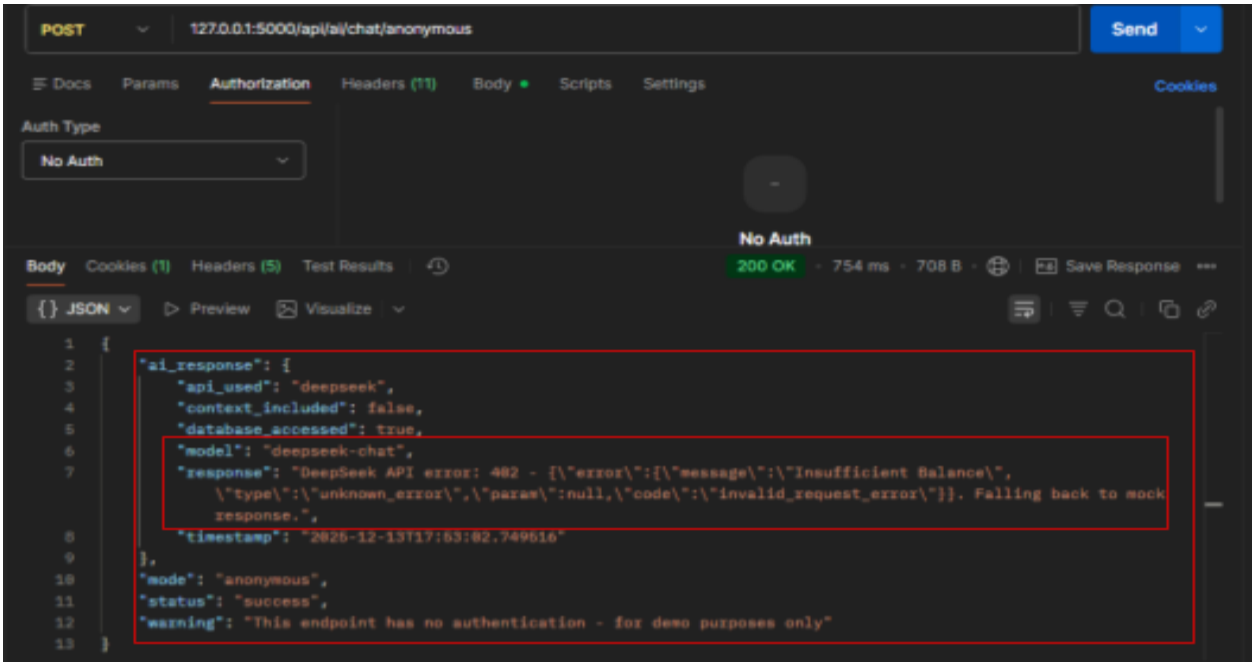
Developers tend to trust and not verify the endpoints that interact with external or third-party APIs, relying on weaker security requirements such as those regarding transport security, authentication/authorization and input validation and sanitization. Attackers need to identify services the target API integrates with (data sources) and, eventually, compromise them

Evidence:

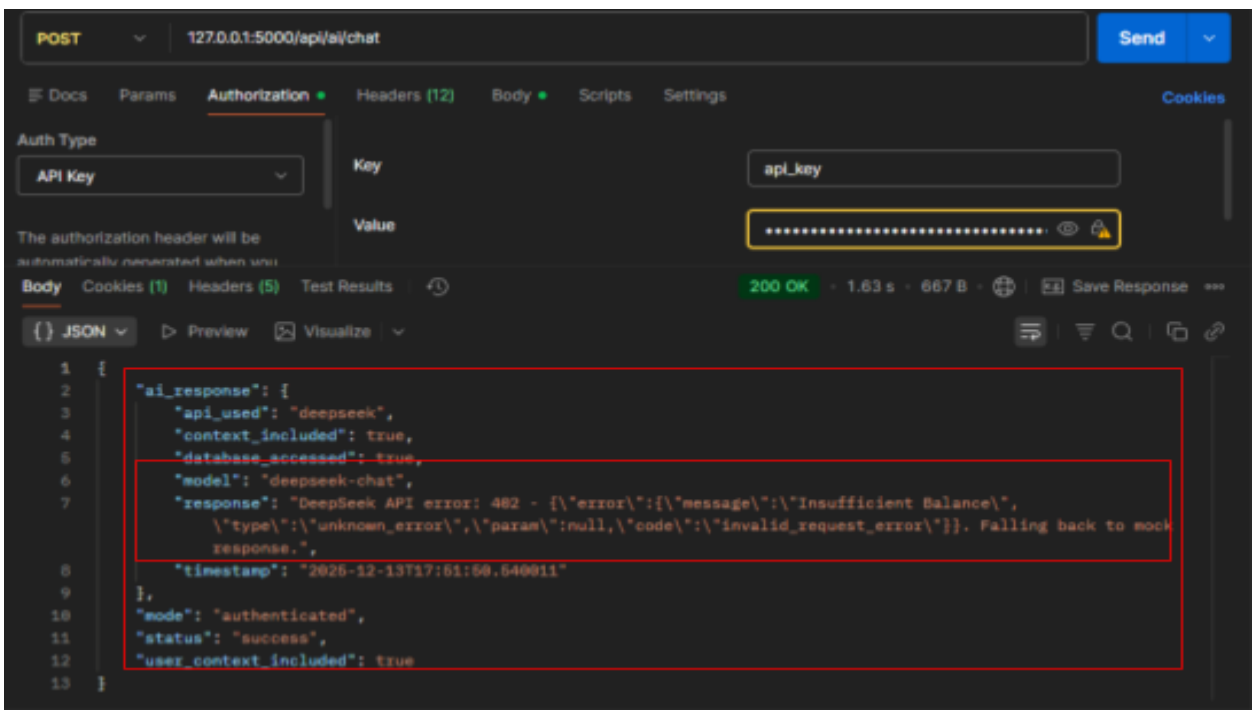
The ***api/aichat*** endpoints expose raw error messages returned from a third-party API (DeepSeek). In this instance, the response discloses a **402 – Insufficient Balance** error originating from the external service, indicating that the backend account has exhausted its usage quota or billing balance.

Instead of handling the error internally, the application **relays the external error message directly to the client** and then proceeds to **fallback logic**, returning a successful response.

1. The ***/api/ai/chat/anonymous*** endpoint



2. The ***/api/ai/chat*** endpoint



Since the API trusts any response from DEEPSEEK, it means that if an attacker was to manipulate the model into exposing sensitive information and they succeed, then the attacker would successfully exploit the Bank API.

Risk

- **Operational Intelligence Leakage**
  Attackers now know:
    - The system relies on a paid third-party service
    - The service has limited balance or quota
    - The API call failed but fallback logic exists
- **Abuse Amplification (API4 Chaining)**
  Knowing that the backend falls back on failure, attackers can:
    - Repeatedly trigger requests without cost to themselves
    - Force the application into degraded or mock mode
    - Hide real service outages
- **Targeted Denial of Wallet (Financial Abuse)**
  If balance was replenished, attackers could intentionally drain it again by sending high-volume requests.
- **Reduced Incident Visibility**
  Returning "status": "success" during failure:
    - Masks outages
    - Breaks monitoring
    - Delays detection of real incidents

Mitigation

- When evaluating service providers, assess their API security posture.
- Ensure all API interactions happen over a secure communication channel (TLS).
- Always validate and properly sanitize data received from integrated APIs before using it.
- Maintain an allowlist of well-known locations integrated APIs may redirect yours to: do not blindly follow redirects.

Finding 8: Path Traversal vulnerabilities

Description

Path Traversal (also known as *Directory Traversal*) is a vulnerability that occurs when an application uses user-supplied input to construct file system paths without proper validation or sanitization.This allows an attacker to manipulate file paths using sequences such as ../ or ..\ to access or write files outside the intended directory, potentially leading to:

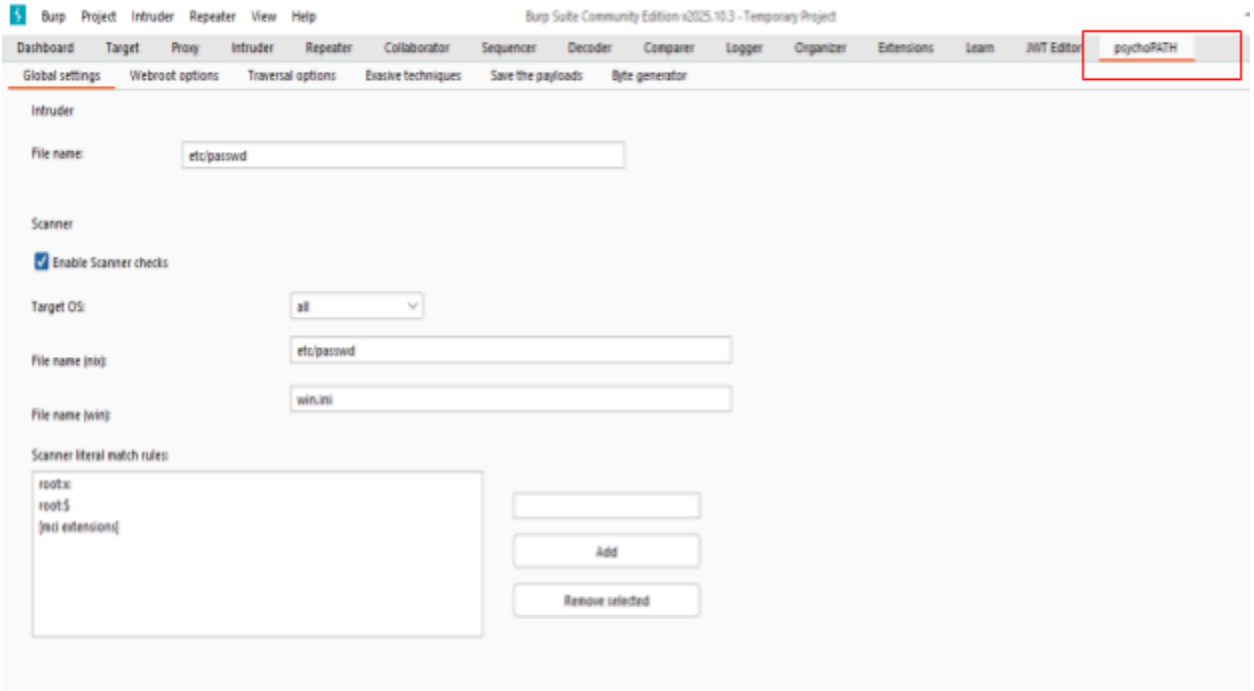
- Unauthorized file access
- File overwrite
- Information disclosure
- Remote Code Execution

Tested endpoint: /upload_profile_picture

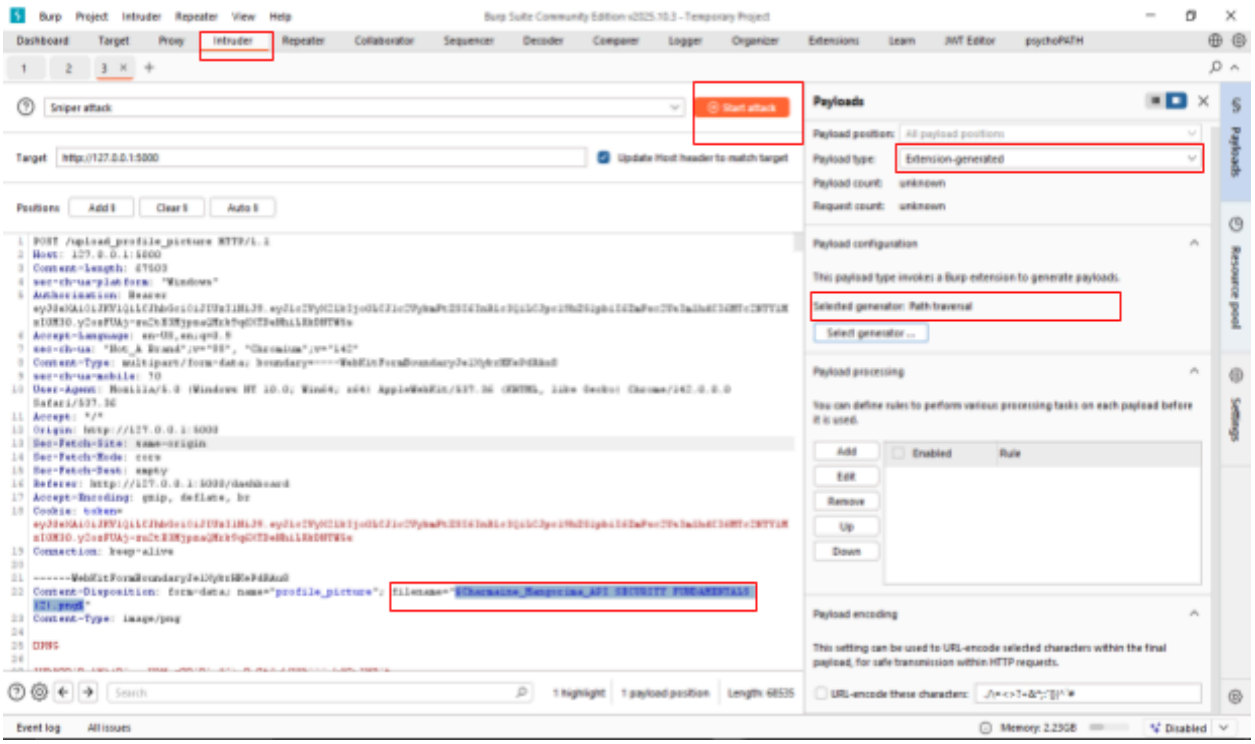
The test targeted was the ***filename*** parameter in the multipart request:

```
1  POST /upload_profile_picture HTTP/1.1
2  Host: 127.0.0.1:5000
3  Content-Length: 67503
4  sec-ch-ua-platform: "Windows"
5  Authorization: Bearer
   eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJlc2VyX2lkIjoOLCJlc2VybmFtZSI6InRlc3QiLCJpcl9hZGlpbiI6ZmFsc2UsImlhdCI6MTc2NTY1M
   zI0M30.y2ozFUAj-zu2tE3MjpzaQMrk9q6XTDeNhiLRkDNTW5s
6  Accept-Language: en-US,en;q=0.9
7  sec-ch-ua: "Not_A Brand";v="99", "Chromium";v="142"
8  Content-Type: multipart/form-data; boundary=----WebKitFormBoundaryJelXykrHKePdRAu8
9  sec-ch-ua-mobile: ?0
10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0
   Safari/537.36
11 Accept: */*
12 Origin: http://127.0.0.1:5000
13 Sec-Fetch-Site: same-origin
14 Sec-Fetch-Mode: cors
15 Sec-Fetch-Dest: empty
16 Referer: http://127.0.0.1:5000/dashboard
17 Accept-Encoding: gzip, deflate, br
18 Cookie: token=
   eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJlc2VyX2lkIjoOLCJlc2VybmFtZSI6InRlc3QiLCJpcl9hZGlpbiI6ZmFsc2UsImlhdCI6MTc2NTY1M
   zI0M30.y2ozFUAj-zu2tE3MjpzaQMrk9q6XTDeNhiLRkDNTW5s
19 Connection: keep-alive
20
21 ------WebKitFormBoundaryJelXykrHKePdRAu8
22 Content-Disposition: form-data; name="profile_picture"; filename="$Charmaine_Mangorima_API SECURITY FUNDAMENTALS
   (2).png$"
23 Content-Type: image/png
24
25 □PNG
26
```

I used the **psycoPATH** extension in Burpsuite that also provides payloads to test path traversals.



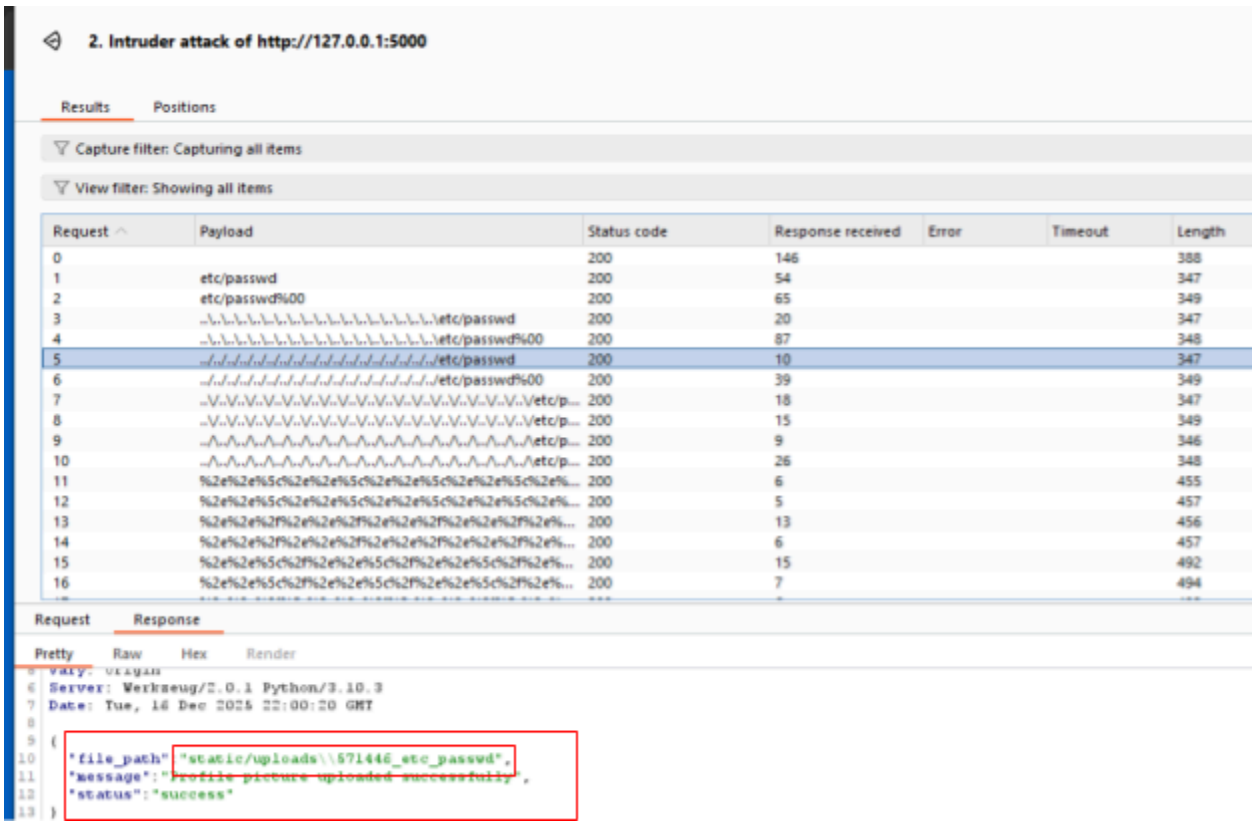
I injected the extension generated path traversal payloads into the ***filename*** field and started the attack using burp intruder:



These payloads were sent using Burp Suite (Intruder and Psychopath-style filename fuzzing) to evaluate whether the application would allow traversal outside the intended upload directory.

Observation

All traversal payloads were successfully uploaded, but the server response showed that the saving directory was hard coded and cannot be bypassed. One of the example response of the 90 paths in the payload tested is below:



Code Review Findings (app.py):

 # Vulnerability: Path traversal possible if filename contains ../

- This comment is incorrect and misleading, as the implemented logic prevents such behavior.
- The use of secure_filename() effectively sanitizes path traversal sequences.

- The upload directory (UPLOAD_FOLDER) is **hard-coded**, preventing directory escape.
- As a result, **path traversal is mitigated at runtime**.

```python
app.py
def upload_profile_picture(current_user):
    try:
        # Vulnerability: No content-type validation
        filename = secure_filename(file.filename)

        # Add random prefix to prevent filename collisions
        filename = f"{random.randint(1, 1000000)}_{filename}"

        # Vulnerability: Path traversal possible if filename contains ../
        file_path = os.path.join(UPLOAD_FOLDER, filename)

        file.save(file_path)

        # Update database with just the filename
        execute_query(
            "UPDATE users SET profile_picture = %s WHERE id = %s",
            (filename, current_user['user_id']),
            fetch=False
        )

        return jsonify({
            'status': 'success',
            'message': 'Profile picture uploaded successfully',
            'file_path': os.path.join('static/uploads', filename)  # Vulnerability: Path disclosure
        })
```

Although path traversal was actively tested against the file upload endpoint using Burp Suite and the Psychopath approach, the vulnerability could not be exploited. Maybe next time I will try other approaches or another endpoint that I might have missed.

Conclusion

This assessment examined the Damn Vulnerable Bank API against the **OWASP API Security Top 10 (2023)**, focusing on **API4–API10**, alongside CORS misconfiguration and path traversal vulnerabilities. Testing conducted using **Postman** and **Burp Suite** revealed multiple security weaknesses, including excessive data exposure through debug functionality, permissive CORS policies allowing untrusted origins to access sensitive responses, the presence of deprecated API versions and improper validation of file path inputs leading to path traversal. These findings demonstrate how insecure configurations and insufficient input validation can expose sensitive data and increase the risk of exploitation. Addressing these issues through secure configuration, strict access control, input validation and removal of legacy and debug functionality is essential to improving the overall security posture of the API.

End of report